

Tranh A Nguyen

June 2026

ELF C++ - 0 protection

Bài này được thực hiện trên kali linux

Tải file từ root-me về (download)

- dùng lệnh file để kiểm tra file --> ELF (rất hợp với linux, PE dành cho Windows)

```
(kali㉿kali)-[~/Downloads]
$ file ch25.bin
ch25.bin: ELF 32-bit LSB executable, Intel i386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, for GNU/Linux 2.6.24, BuildID[sha1]=38a418f97a019e3a6108932d4a7196637e368a88, not stripped
```

- dùng lệnh chmod +x --> linux cho phép file này được chạy (tương tự như chạy exe trên win)

có x ở cuối nè

```
kali@kali:~/Downloads$ ls -l
total 16
-rw-rw-r-- 1 kali kali 12751 Jun 12 05:31 ch25.bin

kali@kali:~/Downloads$ chmod +x ch25.bin

kali@kali:~/Downloads$ ls -l
total 16
-rwxrwxr-x 1 kali kali 12751 Jun 12 05:31 ch25.bin

kali@kali:~/Downloads$
```

./<tên file> chạy thử file

```
kali@kali:~/Downloads$ ./ch25.bin
usage : ./ch25.bin password
```

usage : ./ch25.bin password --> ý là nó đang yêu cầu nhập 1 cái gì đó, cụ thể hơn trong bài nó giống crackme yêu cầu mật khẩu

Nhập linh tinh thử

```
kali@kali:~/Downloads$ ./ch25.bin trunganh
Password incorrect.
```

Tiếp theo ta cần trích xuất các chuỗi ký tự (strings) trong chương trình để tìm đúng vị trí nhập liệu đăng nhập mật khẩu

Ta sẽ sử dụng công cụ “strings” luôn vì nó tiện hoặc có thể đưa vào IDA ftrr nhấn shift +F12

```
kali@kali:~/Downloads$ strings ch25.bin
/lib/ld-linux.so.2
CyIk
libstdc++.so.6
__gmon_start__
_Jv_RegisterClasses
_ITM_deregisterTMCloneTable
_ITM_registerTMCloneTable
__pthread_key_create
_ZNSt4endlIcSt11char_traitsIcEERSt13basic_ostreamIT_0_ES6_
_ZSt4cout
_ZNSaIcED1Ev
_ZNSpLEc
```

Còn nữa cơ nhưng chỉ chụp minh họa thôi

Để tối ưu lại thì mình sẽ dùng thêm “grep” để lọc key

Do là cũng đang tự học luôn nên viết cho bản thân đọc lại nữa nên có thể viết hơi dài dòng

- i là để không phân biệt in hoa in thường

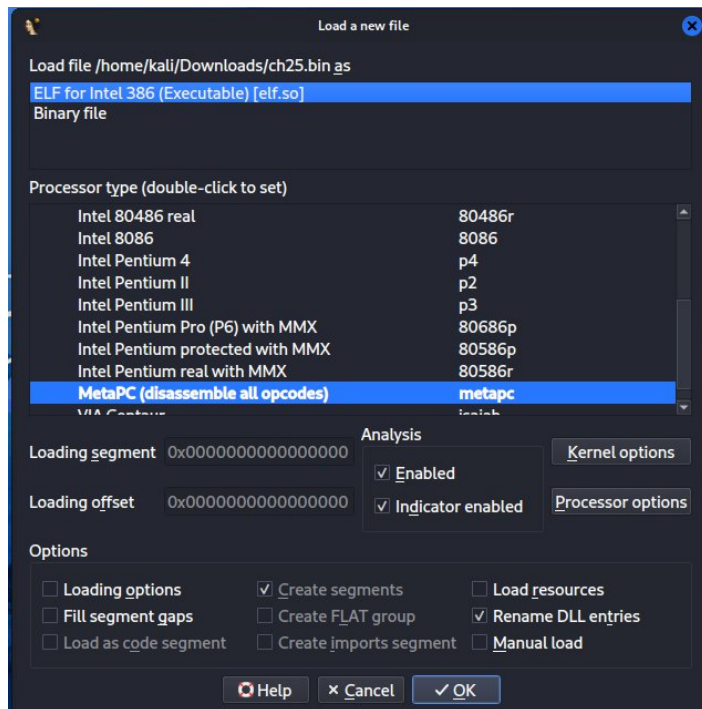
Thông thường khi grep người ta sẽ dùng

grep "flag\\|password\\|secret\\" --> có chứa \

-E (trừ E LỚN) giúp lại bỏ cái \ khiến câu lệnh gọn gàng và dễ đọc hơn

```
kali@kali:~/Downloads$ strings ch25.bin | grep -i -E "flag|valid|correct|wrong|password"
password
Bravo, tu peux valider en utilisant ce mot de passe ...
Congratz. You can validate with this password ...
Password incorrect.
kali@kali:~/Downloads$
```

Đưa file vào IDA Free



Vào function tìm hàm main rồi F5 để có pseudocode C

```

20 p_argc = &argc;
21 if ( argc > 1 )
22 {
23     std::allocator<char>::allocator(&v12);
24     std::string::string(v15, &unk_8048DC4, &v12);
25     std::allocator<char>::allocator(&v11);
26     std::string::string(v14, &unk_8048DCC, &v11);
27     savedregs = v15;
28     v17 = v14;
29     plouf(v13);
30     std::string::~string((std::string *)v14);
31     std::allocator<char>::~allocator(&v11);
32     std::string::~string((std::string *)v15);
33     std::allocator<char>::~allocator(&v12);
34     if ( (unsigned __int8)std::operator==(char)((std::string *)v13, (char *)argv[1]) )
35     {
36         v8 = std::operator<<<std::char_traits<char>>(&std::cout, "Bravo, tu peux valider en
37         std::ostream::operator<<(v8, &std::endl<char, std::char_traits<char>>);
38         v9 = std::operator<<<std::char_traits<char>>(&std::cout, "Congratz. You can validate
39     }
40     else
41     {
42         v9 = std::operator<<<std::char_traits<char>>(&std::cout, "Password incorrect.");
43     }
44     std::ostream::operator<<(v9, &std::endl<char, std::char_traits<char>>);
45     v7 = 0;
46     std::string::~string((std::string *)v13);
47 }
48 else
49 {
50     v3 = *argv;
51     v4 = std::operator<<<std::char_traits<char>>(&std::cerr, "usage : ");
52     v5 = std::operator<<<std::char_traits<char>>(v4, v3);
53     v6 = std::operator<<<std::char_traits<char>>(v5, " password");
54     std::ostream::operator<<(v6, &std::endl<char, std::char_traits<char>>);

```

Để đơn giản và dễ hiểu thì luồng chạy code nó sẽ được giải thích dễ hiểu lại như này:

```

int main(argc, argv) {
    if (argc > 1) {
        string s1 = unk_8048DC4;
        string s2 = unk_8048DCC;

        string result = plouf(s1, s2);

        if (result == argv[1]) {
            print "Bravo...";
            print "Congratz...";
        } else {
            print "Password incorrect.";
        }
    } else {
        print "usage : ./binary password";
        return 5;
    }
}

```

Ta có:

- Có 2 chuỗi hardcoded trong binary (unk_8048DC4 và unk_8048DCC)
- Cả 2 chuỗi này được đưa vào hàm plouf
- Kết quả của plouf được so sánh trực tiếp với argv[1] (password người dùng nhập)

=> bài cho 2 chuỗi cố định → đưa vào hàm X → kết quả so sánh với input user

Tức là kết quả của hàm X chính là password, vì input của X không phụ thuộc user (ý là đề đã có sẵn key và mình cần tìm đúng vậy thôi)

Giờ chỉ cần cho debugger chạy rồi hỏi lại máy tính kết quả là ok

```

[Dữ liệu A] + [Dữ liệu B] ---> [hàm plouf] ---> [Kết quả]
                                     |
                                     v
                               So sánh với [password bạn gõ]
                                     |
                               Giống → "Bravo"
                               Khác  → "Password incorrect"

```

Hàm plouf nó đang biết đổi 2 dữ liệu đưa vào, có thể dùng ida để xem thuật toán các người viết làm nhưng đối với bài này thì không cần. Tại sao ư ?

Vì dữ liệu là dữ liệu có sẵn, bài cho, nó không phải là thứ user nhập

Cho nên dù plouf làm gì phức tạp, kết quả luôn ra một con số/chuỗi cố định tương tự như casio 570 vn plus :V
bấm 2 + 2 luôn ra 4, không cần hiểu máy tính bên trong làm sao, vì nó có biết như nào thì cũng chỉ có 1 kết quả duy nhất và mình chỉ cần gdb chạy là xong, không cần hiểu thuật toán.

Tiếp tục dùng gdb để debugger

```
kali@kali:~/Downloads$ gdb -q ./ch25.bin
pwndbg: loaded 198 pwndbg commands. Type pwndbg [filter] for a list.
pwndbg: created 12 GDB functions (can be used with print/break). Type help function to see them.
Reading symbols from ./ch25.bin...
(No debugging symbols found in ./ch25.bin)
----- tip of the day (disable with set show-tips off) -----
Use GDB's dprintf command to print all calls to given function. E.g. dprintf malloc, "malloc(%p)\n"
id*)$rdi will print all malloc calls
```

Cách 1: Lấy địa chỉ trong IDA

```
.text:08048B4C    ; try {
.text:08048B4C 00C call     _Z5ploufSsSs ; plouf(std::string,std::string)
.text:08048B4C    ; } // starts at 8048B4C
```

Cách 2: xem địa chỉ trong gdb

```
pwndbg> disassemble main
```

```
0x08048b49 <+195>:  mov     DWORD PTR [esp],eax
0x08048b4c <+198>:  call    0x0804898d <_Z5ploufSsSs>
0x08048b51 <+203>:  sub     esp,0x4
```

Địa chỉ cần đặt breakpoint ngay tại vị trí hàm plouf trong IDA

Run giá trị linh tinh

```
pwndbg> break *0x08048b4c
Breakpoint 1 at 0x08048b4c
pwndbg> run trunganhdeptraoi
Starting program: /home/kali/Downloads/ch25.bin trunganhdeptraoi
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
```

Cách đặt endpoint ?

Hiện tại tao không rõ lí thuyết lắm nhưng hiểu đơn giản thì nếu Muốn xem kết quả trả về của một hàm → đặt breakpoint tại địa chỉ ngay sau lệnh call của hàm đó, vì đó là điểm đầu tiên EAX chứa giá trị trả về.


```
pwndbg> break *0x8048b51
Breakpoint 2 at 0x8048b51
pwndbg> continue
Continuing.
```

Vì bài này EAX đang là điểm đầu tiên nên mình chỉ cần quan tâm tới EAX khi debug (câu này quan trọng nhé vì nó liên quan tới cái sau)

Pwndbg nó mạnh vl :))))

Sau khi endpoint 2 và continue thì ta sẽ nhìn vào stack vì dữ liệu của thanh ghi sẽ được đẩy lên đó.

Mà cái thằng pwndbg nó hỗ trợ tốt vô cùng :))

giúp mình khỏi phải tự dereference con trỏ bằng tay (x/s *(int*)...)

nó tự "đi theo" con trỏ và in sẵn kết quả của string.

Nhưng mình vẫn cần biết tìm đúng dòng

Đó là lí do tại sao có dòng EAX nói ở trên

Tìm đúng EAX trong stack --> key của bài này

```

[ STACK ]
00:0000 | esp 0xffffd1d4 -> 0xffffd1e8 -> 0x804f41c -> 0xaf67b350
01:0004 | -020 0xffffd1d8 -> 0xffffd1ec -> 0x804f3fc -> 0xca15d618
02:0008 | -01c 0xffffd1dc -> 0xf7cb1440 -> 0xf7cb2420 -> 0
03:000c | -018 0xffffd1e0 -> 2
04:0010 | eax 0xffffd1e4 -> 0x804f50c -> 'Here_you_have_to_understand_a_little_C++_stuffs'
05:0014 | edx 0xffffd1e8 -> 0x804f41c -> 0xaf67b350
06:0018 | -00c 0xffffd1ec -> 0x804f3fc -> 0xca15d618
07:001c | -008 0xffffd1f0 -> 0xffffd210 -> 2

```

pwndbg giảm bớt công sức tính toán địa chỉ, nhưng không thay thế việc hiểu register nào chứa gì, đó vẫn là kiến thức cần có

Bonus:

Sau khi plouf chạy xong (ret về), kết quả trả về được đặt vào register EAX.

Với bài này, mình chỉ cần quan tâm **giá trị của EAX** tại thời điểm đó.

Xác minh kết quả:

```
pwndbg> quit
kali@kali:~/Downloads$ ./ch25.bin Here_you_have_to_understand_a_little_C++_stuffs
Bravo, tu peux valider en utilisant ce mot de passe...
Congratz. You can validate with this password...
kali@kali:~/Downloads$
```

DONE!

Bài học:

- Mục đích của bài này muốn nói là đọc pseudocode C để hiểu luồng chạy và tư duy của code. Nó có cho sẵn 2 giá trị từ bài rồi nên không cần phải áp lực đi giải thuật toán (giải mã) mà chỉ cần gdb chạy rồi xem kết quả. Còn sau này mà làm bài khó thì giá trị do người dùng nhập thì lúc đó phải viết python giải mã thuật toán mã hóa để tìm key, lúc đó hiểu luồng code là chuyện bắt buộc.
- Muốn xem kết quả trả về của một hàm → đặt breakpoint tại địa chỉ ngay sau lệnh call của hàm đó, vì đó là điểm đầu tiên EAX chứa giá trị trả về. và cái thanh ghi (register) trả giá trị về rất quan trọng vì nó dùng để debug sau này

Bonus:

Sự bá đạo của pwndbg và gef:

gdb thuần khi dừng tại breakpoint không tự hiển thị gì
 --> phải tự gõ lệnh x/s, print, và phải tự tính địa chỉ qua nhiều lớp dereference
 (*(int*)(*(int*)(\$esp+4)))

Nhưng có tí plugin vào nó đỡ mệt hẳn

pwndbg/gef khi dừng tự động in ra:

- Toàn bộ registers, kèm theo tự dereference và hiển thị nội dung cuối (string, nếu có)
- Stack, kèm chú thích register nào trỏ tới đâu
- Disassembly xung quanh vị trí hiện tại
- Backtrace

IDA đủ khi:

- Bài chỉ cần đọc hiểu logic (if/else, so sánh, vòng lặp) để biết "điều kiện pass là gì"
- Dữ liệu cần (string, số, key...) nằm sẵn, rõ ràng trong .rodata và IDA hiển thị đúng, đọc được trực tiếp
- Không có giá trị nào được tính toán tại runtime (mọi thứ là hằng số tĩnh)

Cần thêm debugger khi:

- Pseudocode IDA bị lỗi/thiếu (như cảnh báo "bad sp value" ở bài này) → không tin được hoàn toàn
- Giá trị cần biết là kết quả của một phép tính (như `plouf()` ở đây) — IDA chỉ cho biết "có phép tính gì", không cho biết "kết quả là bao nhiêu" một cách trực quan
- Cấu trúc dữ liệu phức tạp (như `std::string`) khiến việc đọc raw bytes trong `.rodata` dễ tính sai offset/length
- Bài có anti-debug, encryption, obfuscation — cần chạy thử để xem hành vi thật, không thể chỉ đọc static